# Command sheet วันสอบ — demo บน cloud (CT141)

> อิงจากการซ้อมจริง 2026-10-09 บน CT141 (ผ่านทุกขั้น) · แผนสำรองถ้า cloud ล่ม = stack บนเครื่อง ดู demo-cloud.md ข้อ 5
> สัญลักษณ์: **[minner]** = เทอร์มินัลบนเครื่องเรา · **[CT141]** = เทอร์มินัลที่ SSH เข้า CT141 แล้ว (user `logchain`, `~/logchain`) · ✋ **ทำมือ** = ต้องทำเองในเบราว์เซอร์/เทอร์มินัล · 🔐 = sudo หรือ `read -s` → รันในเทอร์มินัลตัวเอง ไม่ใช่ผ่าน `!` ของ Claude Code
> ห้าม: `demo-start.sh` · `docker compose down -v` · reboot CT141 ระหว่าง tamper ① ถึง ③ · resolve alert ด้วย SQL (ไม่มี audit)
> ⛔ **ห้ามรัน `./scripts/demo-start.sh` เด็ดขาด (local และ CT141)** — สคริปต์ยุคก่อน container · `pkill -f "node dist/main"` ฆ่า **backend ใน container** ได้ เพราะ node ใน container รันเป็น uid 1000 = uid ของเรา (local `chanakan` · CT141 `logchain`) → backend ปิด HTTP + DB แต่ process ไม่ตาย Docker ไม่ restart ให้ → `/health` ไม่ตอบ · ไม่มี log/seal/alert จนกว่าจะ restart (เกิดจริงบน local 2026-10-09 19:03) · ข้อความท้ายสคริปต์ "ต่อด้วย demo-preflight.sh" ก็ล้าสมัย — ใช้ขั้น 2 ของชีตนี้แทน · ถ้าเผลอรัน: `HOST_UID=$(id -u) HOST_GID=$(id -g) docker compose restart backend` แล้วเช็ค `/health` ต้องได้ `"connected":true`

สีในบล็อกคำสั่ง: ✋ ทำมือ | 🔐 sudo / เทอร์มินัลตัวเอง | ⏳ จุดที่ต้องรอ | # ผลที่ควรเห็น

## 0) [minner] ก่อนขึ้นเวที ~30 นาที — เปิดหน้าต่าง/แท็บให้พร้อม

```
# 🔐 [minner] เปิด VPN (sudo — เทอร์มินัลตัวเอง)
sudo wg-quick up wg-friend
# ✋ เบราว์เซอร์: https://logchain.nareubad.work → login admin-user (รหัส + OTP)
#    เปิดค้างไว้ 2 แท็บ: หน้า Alerts · หน้า Chain Integrity
# [minner] API สาธารณะต้องได้ 200 (403 = โดน Cloudflare challenge — ดู demo-cloud.md ข้อ 4)
curl -s -o /dev/null -w '%{http_code}\n' https://logchain-api.nareubad.work/health
```

## 1) [minner] → SSH เข้า CT141 (เปิดเทอร์มินัลนี้ค้างไว้ทั้ง demo)

```
ssh -i ~/.ssh/nblab_ct141 -o IdentitiesOnly=yes root@172.16.40.141
su - logchain
cd ~/logchain
```

## 2) [CT141] เช็คความพร้อม

```
date -u; curl -sI https://www.google.com | grep -i '^date'
#   → สองเวลาต่างกันไม่เกินไม่กี่วินาที (ซ้อม: 1 วิ)
docker exec logchain-kafka-1 sh /opt/logchain/kafka-cli.sh \
  kafka-topics.sh --describe --under-replicated-partitions
#   → ต้องไม่มีผลลัพธ์ (ว่าง)
HOST_UID=$(id -u) HOST_GID=$(id -g) docker compose ps --format '{{.Service}}\t{{.Status}}'
#   → Up ครบ 21 service · ตัวที่มี health check ต้อง (healthy)
curl -s localhost:3000/health; echo
#   → มี "connected":true

# [CT141] ยอด wallet sealer (preflight ไม่ตรวจ — เงินหมด = anchor ล้มเงียบ ๆ ทุกนาที)
#   address แยกไว้ในตัวแปรสั้น ๆ — บรรทัดยาวโดนตัดตอนก๊อป = address พัง → .result เป็น null
W=0xA03e739e622Cd8D6D65b9B36ea226B17EE661C59
RPC=$(grep -E '^BLOCKCHAIN_RPC_URL=' .env | cut -d= -f2- | tr -d '\r')
Q="{\"jsonrpc\":\"2.0\",\"method\":\"eth_getBalance\","
Q="$Q\"params\":[\"$W\",\"latest\"],\"id\":1}"
curl -s -X POST -H 'Content-Type: application/json' --data "$Q" "$RPC" \
  | jq -r '.result // .error' | python3 -c 'import sys
v = sys.stdin.read().strip()
print("POL", int(v, 16) / 1e18) if v.startswith("0x") else print("RPC error:", v)'
#   → "POL 0.0…" · ถ้าขึ้น "RPC error" ให้เช็คว่า W= ลงท้าย …661C59 ครบ
#   เกณฑ์: ต้อง ≥ 0.05 POL ก่อนเริ่ม demo (≈ 12 tx ที่เพดาน = 3 รอบ all-rules แบบแย่สุด)
#     1 รอบ all-rules = 2–4 tx (รวดเดียว 2 · STEP=1 ช้า ๆ 3–4
#       เพราะ log กระจายไปหลายนาทีของ cron)
#     1 tx = 78,156 gas · base fee Amoy ≈ 0 → ค่า gas เกือบทั้งหมดคือทิป (priority fee)
#       ทิปถูกกดไว้ไม่เกิน BLOCKCHAIN_MAX_PRIORITY_FEE_GWEI=50 (35013ad)
#       → 30 gwei ≈ 0.0023 POL · เพดาน 50 gwei ≈ 0.0039 POL ต่อ tx
```

```
#     แย่สุด 4 tx × 50 gwei ≈ 0.016 POL/รอบ · tamper / mtls / resolve = 0 tx
#     ⚠ ไม่มีเพดาน = ใช้ทิปที่ RPC แนะนำตรง ๆ: 10-11 แนะนำ 500 gwei → 0.039 POL/tx (≈ 0.16 POL/รอบ)
#   ต่ำกว่าเกณฑ์: เติม POL (Amoy testnet) เข้า $W แล้วรันบล็อกนี้ซ้ำ
# [CT141] เพดานทิปต้องเปิดอยู่
docker compose logs backend 2>&1 | grep -o 'maxPriorityFee=[^ ]*' | tail -1
#   → "maxPriorityFee=50" · ขึ้น "uncapped" หรือไม่ขึ้นเลย = .env ไม่มี
#     BLOCKCHAIN_MAX_PRIORITY_FEE_GWEI=50 → ใส่แล้ว docker compose up -d --no-deps backend
# หลัง demo ดูทิปที่ใช้จริงต่อ tx: docker compose logs backend | grep "Priority fee"
# PSQL = ทางลัดไว้ใช้ต่อในขั้น 5 ด้วย
PSQL() { docker exec logchain-postgres psql -U logchain -d logchain -tAc "$1"; }
PSQL "select status, count(*) from batches group by 1"
#   → มี CONFIRMED · FAILED ~1,161 ใบเดิม (ใบเปล่าจากช่วงเงินหมด 10-08) ปกติ ไม่ต้องแตะ
```

## 3) [CT141] ขอ token ของ log-ingestor (อายุ 15 นาที — ขอใหม่ถ้าค้างนาน)

```
set -a; . ./.env; set +a
KC="${KEYCLOAK_LOCAL_URL:-http://localhost:${KEYCLOAK_HOST_PORT:-8080}}"
T=$(curl -s -d grant_type=client_credentials -d client_id=log-ingestor \
      --data-urlencode client_secret="$LOGCHAIN_INGESTOR_SECRET" \
      "$KC/realms/logchain/protocol/openid-connect/token" \
  | jq -r .access_token)
echo "${#T}"
#   → ตัวเลขราว 1000+ (ความยาว token) · ได้ 4 (= null) หรือ 0 = ขอ token ไม่สำเร็จ
```

## 4) [CT141] กฎครบ 9 ข้อ — ✋ สลับไปดูแท็บ Alerts ระหว่างรัน

```
STEP=1 ./scripts/demo-all-rules.sh "$T"
#   STEP=1 = กด Enter ทีละข้อ (ตัด STEP=1 ออก = รันรวดเดียว ~50 วิ)
#   → แต่ละข้อ HTTP 201 + "✓ alert rule <id> ขึ้นแล้ว" · กลุ่มควบคุมไม่มี alert
#   → จบด้วย "✓ ครบ 9 rule"
#     CRITICAL: 5710 5820 31100 31151 60002 90001 · WARNING: 5715 60001 90002
#   → อีเมลแจ้งเตือน CRITICAL ออก ~6 ฉบับ
#   จด source ที่บรรทัดแรกพิมพ์ (rule-demo-HHMMSS) ไว้ใช้ตอน cleanup
```

## 5) [CT141] tamper — ✋ เปิดแท็บ Chain Integrity + Alerts

```
./scripts/demo-tamper.sh no-admin-token
#   → "IMMUTABLE_LOG: UPDATE not permitted on logs table" (trigger บล็อก UPDATE ตรง)
#   → "แก้ hash แล้ว" · verify-now ได้ 401 (ตั้งใจ — ให้ cron ตรวจแทน)
B=$(awk '{print $3}' /tmp/logchain-tamper-backup.txt)
# ⏳ รอ cron verify รอบที่ 1 (≤ 1 นาที · ซ้อม ~25 วิ) — รันซ้ำจนขึ้น TAMPERED
PSQL "select status from batches where id='$B'"
#   → TAMPERED · หน้า Alerts มี INTEGRITY_TAMPERED CRITICAL
#     Chain Integrity ตกจาก 100% · อีเมล 1 ฉบับ
#   PSQL หาย (เปิดเทอร์มินัลใหม่) → รันบรรทัด PSQL() ในขั้น 2 ซ้ำ

./scripts/demo-tamper.sh no-admin-token restore
#   → "⚠ … ยังเป็น TAMPERED … ไม่ resolve alert" + exit 1 = ปกติ (คืน hash แล้ว รอ cron)
# ⏳ รอ cron verify รอบที่ 2 (≤ 1 นาที) — รันซ้ำจนขึ้น CONFIRMED
PSQL "select status from batches where id='$B'"
#   → CONFIRMED · alert tamper ถูก backend ปิดเอง (RESOLVED)

./scripts/demo-tamper.sh no-admin-token restore
#   → "กู้คืนแล้ว — batches: … CONFIRMED x…"
#   → "backend ปิดเองตอน verify อีก 1, ยังค้าง OPEN 0"
#   → ไฟล์สำรองถูกลบ · ห้ามข้ามขั้นนี้ (ไม่งั้น Chain Integrity ค้างต่ำกว่า 100%)
```

## 6) [CT141] mTLS — อ่านอย่างเดียว ~8 วิ

```
./scripts/demo-mtls.sh
#   1) → มีข้อความต้อนรับ Bitnami หลายบรรทัด (ปกติ) แล้วรายชื่อ topic
#        alerts.cde · alerts.raw · logs.raw · logs.raw.dlq → "✅ เข้าถึงได้"
#   2) → SslAuthenticationException … bad_certificate → "✅ ถูกปฏิเสธ = mutual TLS ทำงาน"
```

```
#   3) → "(เปิด backend+detection ด้วย KAFKA_SSL_ENABLED=true …)" — ข้าม/อธิบาย:
#        backend/detection คุย Kafka ผ่าน DOCKER_SSL :9094 ใน docker network
#        ซึ่ง ss บน host มองไม่เห็น
#   4) → CN = kafka-1 · SAN kafka-1, localhost, 127.0.0.1 · หมดอายุ 2029-01-03

exit   # ออกจาก logchain
exit   # ออกจาก CT141 → กลับมา minner
```

### 7) [minner] cleanup — ปิด alert ของ rule-demo-* ผ่าน API (มี audit_access)

```
# ✋ แท็บ dashboard (admin-user) → DevTools → Network → request ไป logchain-api
#    → คัดลอกค่าหลัง "Bearer " ของ Authorization header
#    ⚠ token อาจเหลืออายุแค่ 1–2 นาที (ซ้อม: 73 วิ) — refresh หน้าแล้วก๊อปสด
#    ⚠ ทำคำสั่งข้างล่างต่อกันทันที
# 🔐 [minner] zsh — รันในเทอร์มินัลตัวเอง (ไม่ใช่ `!` · `read -p` ใช้ใน zsh ไม่ได้)
umask 077; read -rs "TOK?token: " && printf '%s' "$TOK" > ~/.op.token
unset TOK; echo; wc -c ~/.op.token
#   → ~1400 ไบต์ · 0 = วางไม่ติด

TOK=$(cat ~/.op.token); API=https://logchain-api.nareubad.work/api/v1
for id in $(curl -sf -H "Authorization: Bearer $TOK" $API/alerts \
  | jq -r '.[] | select((.source|startswith("rule-demo-")) and .status=="OPEN") | .id'); do
  curl -s -o /dev/null -w "$id %{http_code}\n" -X PATCH \
    -H "Authorization: Bearer $TOK" "$API/alerts/$id/resolve"
done
#   → ทุกบรรทัดลงท้าย 200 (ซ้อม: 200 ×9 · 200 ×18)
#   ไม่มีบรรทัดเลย = token หมดอายุ (curl -f เงียบ) → ก๊อปใหม่แล้วรันซ้ำ
#   alert ของ tamper ไม่ต้องปิด — backend ปิดเองแล้ว
#   alert ของ source อื่น (11 ใบเดิม) ไม่แตะ
rm -f ~/.op.token; unset TOK
```

### 8) [minner] ปิด VPN

```
# 🔐 sudo — เทอร์มินัลตัวเอง
sudo wg-quick down wg-friend
```

## ถ้าเจอปัญหา

| อาการ | ทำอะไร |
|---|---|
| `POST /logs` ค้าง / alert ไม่ขึ้น ทั้งที่ HTTP 201 | นาฬิกาเพี้ยน → `demo-cloud.md` ข้อ 6 (cloud: แจ้งเพื่อนตรวจ NTP ของ Proxmox) |
| `✗ ไม่เห็น alert rule …` บางข้อ | ดู `docker logs --since 5m logchain-detection-consumer` · ขอ token ใหม่แล้วรัน all-rules ซ้ำ (source ใหม่อัตโนมัติ) |
| batch ไม่กลับเป็น CONFIRMED หลัง restore | รออีก 1 นาทีแล้วรัน `restore` ซ้ำ — ไฟล์สำรองยังอยู่ที่ `/tmp/logchain-tamper-backup.txt` |
| log ของ backend มี `insufficient funds` | เงินหมด → เติม POL เข้า `0xA03e739e622Cd8D6D65b9B36ea226B17EE661C59` · log ที่ค้างจะ seal เองรอบถัดไป (batch ที่ล้มระหว่างนั้นเป็น FAILED ใบเปล่า ไม่ต้องแตะ) |
| เช็คยอดได้ `null` / `invalid literal for int()` | address ใน JSON ถูกตัดบรรทัดตอนก๊อป → ใช้บล็อก `W=` ในขั้น 2 และก๊อปจากไฟล์ `.md` ไม่ใช่ PDF |
| backend `Up (unhealthy)` · `/health` ไม่ตอบ · log มี `Driver not Connected` | มักเกิดจากเผลอรัน `demo-start.sh` (มีไฟล์ `/tmp/logchain-backend.log`) → `HOST_UID=$(id -u) HOST_GID=$(id -g) docker compose restart backend` · `/health` ต้อง `"connected":true` |
| dashboard "Could not load dashboard data" | `curl https://logchain-api.nareubad.work/health` ต้องได้ 200 ไม่ใช่ 403 (Cloudflare) |
| cloud ล่มทั้งหมด | ใช้ stack บนเครื่อง (`https://localhost:3453`) — สคริปต์ชุดเดียวกัน · `./scripts/demo-preflight.sh` ก่อน |